Learning Visibly Pushdown Languages under Rewriting Rules

Jan Otop
University of Wrocław

Piotr Traczyński
University of Wrocław

Abstract

We present a method for approximating Visibly Pushdown Languages (VPLs) using string rewriting rules. We apply this method to extend the Angluin-style active learning framework for VPLs by incorporating deductive inference from rewrite rules. String rewriting rules encode prior knowledge about the target language and the learning algorithm can deductively infer answers to some queries without asking the oracle, thereby reducing its query complexity. We demonstrate the efficacy of this approach through examples of rewrite systems that capture natural properties of structured data and significantly reduce the number of equivalence queries directed to the oracle. Finally, we address the rule synthesis problem, which is a first step for making active automata learning algorithms explainable.

2012 ACM Subject Classification Theory of computation → Grammars and context-free languages; Theory of computation → Rewrite systems

Keywords and phrases visibly pushdown automata, string rewriting, active learning

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Active automata learning. The aim of *active automata learning* is to generate a *deterministic finite automaton (**DFA**)* recognizing an unknown regular language $\mathcal{L}$ (target language) given access to an oracle that answers queries about $\mathcal{L}$. There are two types of queries: *membership* of a word in $\mathcal{L}$, and *equivalence* of the language of a given **DFA** $\mathcal{A}$ with the target language $\mathcal{L}$, where the oracle either confirms equivalence or returns a *counterexample* word witnessing the difference between $\mathcal{L}$ and $\mathcal{L}(\mathcal{A})$. This framework was proposed in the seminal paper [5] together with the L*-algorithm, which enables learning in polynomial time. Active automata learning has been extensively researched for both efficiency [28, 47] and extensions to other automata-based models such as nondeterministic finite automata [15], tree automata [21], nominal automata over infinite alphabets [42] and weighted automata [10].

Learning context-free languages. The active learning framework has been extended to pushdown languages [32, 6, 20, 9, 35], where the representations typically considered have been context-free grammars. However, generating context-free grammars from queries is substantially more difficult than generating **DFA**, as there is no counterpart to the Myhill–Nerode theorem for pushdown languages. Furthermore, the equivalence problem for context-free grammars is undecidable, and for deterministic context-free grammars, which are equivalent to deterministic pushdown automata, it is decidable, but the best currently known upper bound is primitive recursive [44, 29]. However, a subclass of context-free languages called *visibly pushdown languages (**VPLs**)* [39, 3], where the stack operations are determined by the input letters, admits a Myhill–Nerode-style characterization [2] and language equivalence of deterministic visibly pushdown automata (**DVPA**) is decidable in polynomial time [3].

© Jan Otop and Piotr Traczyński;
licensed under Creative Commons License CC-BY 4.0

Learning VPLs. While **DVPA** share multiple good properties with **DFA** such as polynomial-time closure over Boolean operations and subset-based determinization, the minimization problem for **DVPA** is NP-complete [25], which rules out polynomial-time active learning algorithms, unless P=NP. There are different approaches to address this problem including restricting the learning problem to subclasses of **DVPA** [36], working only with **DVPA** in a special form such as SEVPA, MeVPA or CDA [27, 17, 31], which may involve exponential blow-up upon transformation to **DVPA** [2], or augmenting the oracle with queries that depend on the structure of the target **DVPA** [40].

Applications of learning VPLs. Learning **VPLs** has been applied to a wide range of domains, including inference of document-validation specifications from JSON schemas, generation of valid program inputs [9, 31], and verification and explainability of Recurrent Neural Networks (RNNs) [8]. In these applications, membership queries can be implemented rather straightforwardly, but equivalence queries are typically approximated because exact equivalence checking is often infeasible or undecidable. For instance, verifying the equivalence of a **DVPA** against a general program is undecidable, while comparison with an RNN inevitably yields a negative result, as standard RNNs are limited to recognizing regular languages. Furthermore, the non-equivalence witness can be prohibitively large [26], making the negative answer meaningless. Recently, Large Language Models (LLMs) have been employed to implement the teacher [38, 13, 23]. LLMs can only answer membership queries; again, equivalence queries are merely approximated by posing a large number of membership queries. Consequently, reducing the number of equivalence queries is a key challenge in active automata learning, which this paper addresses through the *advice* mechanism.

Learning with advice. We study the active learning framework augmented with advice [24], where the learning algorithm is provided with *advice* expressing prior knowledge regarding the target language. Given advice, we run an active learning algorithm and intercept equivalence queries to first check whether a candidate automaton is *consistent* with the given advice. If it is not, we conclude that it is not equivalent; otherwise we forward the candidate automaton to the oracle. By using the advice to answer queries deductively, this framework bridges two synthesis paradigms: deductive synthesis from specifications and inductive synthesis via queries. This hybrid approach allows a language to be represented partially through queries and partially through advice expressed as a *string rewriting system (SRS)*. Since SRSs and automata operate on fundamentally different principles, certain language properties admit far more succinct descriptions as rewriting rules than they do as a sequence of query-answer interactions.

String rewriting rules for pushdown languages. A String Rewriting System (SRS) consists of rewrite rules $l \to r$, where l, r are words. *Rewriting* is the process of iteratively transforming words by replacing the infix that matches the left-hand side of a rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with its corresponding right-hand side $\boldsymbol{\beta}$. For instance, commutativity of letters a, b can be expressed with $ab \to ba$, which may require multiple query-answer interactions to express. However, if a is a call letter and b is a return letter, the commutativity rule disrupts the matching between call and return letters. We discuss this issue in the paper and define classes of *well-matched* and *matching-preserving* rules, which are compatible with the implicit structure of pushdown languages. Furthermore, a simple property "all content between the opening comment tag and the corresponding closing tag can be discarded" is not expressible by simple rewrite rules, as it involves matching of the opening and closing tags. We address this issue by introducing rewrite rules with variables, which can be substituted with *well-matched* words.

Consistency of an SRS with a Language. In our framework, an advice SRS induces

an equivalence relation on words relative to the target language $\mathcal{L}$. Specifically, if a word w belongs to $\mathcal{L}$, then every word obtained by rewriting w must also be in $\mathcal{L}$; symmetrically, rewriting must respect the complement of $\mathcal{L}$. We also consider one-sided variants based on implication stating that: (1) words in the language may only be rewritten to words in the language, while there are no restrictions for words outside the language, or (2) words outside the language may only be rewritten to words outside the language. This allows the SRS to capture high-level structural properties, such as commutativity or idempotence, without necessarily characterizing the learned language completely.

Contributions. While active automata learning with advice was introduced in [24], extending this framework from regular languages to **VPLs** and Deterministic Visibly Pushdown Automata (DVPAs) is technically demanding. This transition requires addressing structural challenges that are absent in the regular case. The main contributions presented in this paper are:

1. We define *well-matched* and *matching-preserving* rules that are compatible with the structural properties of pushdown languages. Furthermore, we extend these rules with (context) variables in order to capture natural properties of pushdown languages.
2. We develop algorithms that infer answers to equivalence queries for **DVPA**.
3. We establish the complexity of the rule synthesis problem. Since it is a functional problem, we instead show that a related decision problem is NP-complete.
4. We demonstrate the practical benefit of the advice mechanism through experiments.

Related work. There is a large body of work on active learning of context-free grammars [32, 46, 4, 43, 45, 20, 9, 11, 35, 50], as well as on learning visibly pushdown grammars and visibly pushdown automata [27, 40, 17, 8, 31, 22]. These works primarily aim to improve learning algorithms for arbitrary input languages by exploring alternative representation models, augmenting the learning framework (e.g., oracles can answer additional types of queries, or the algorithm is given the set of non-terminals), or altering the mode of learning. These works address different aspects of the problem and are largely independent of our approach. Unlike these works, our approach restricts the search space of the learning algorithm. A similar approach is specializing the Angluin's algorithm to subclasses of languages in order to improve the performance of learning algorithms. A related approach involves specializing Angluin's algorithm to subclasses of languages to improve learning performance, studied for Mealy machines [1, 12, 37, 33, 41] and tree automata [48, 14].

2 Preliminaries

Pushdown alphabet. A pushdown alphabet $\widehat{\Sigma}$ consists of three disjoint sets of letters $\widehat{\Sigma} = (\Sigma_c, \Sigma_l, \Sigma_r)$ called, respectively, *call*, *local* and *return* letters.

Well-matched words. The set of well-matched words over a pushwdown alphabet $\widehat{\Sigma}$ is the least set containing ϵ such that (1) if w is well-matched, c is a call letter and r is a return letter, then cwr is well-matched, (2) if w is well-matched and l is a local letter, then lw and wl are well-matched, and (3) if w_1, w_2 are well-matched, then the concatenation $w_1 w_2$ is well-matched. In a word w (not necessarily well-matched), if $w = x\mathbf{c}y\mathbf{e}z$ such that y is a well-matched word, we say that letters $\mathbf{c}$ and $\mathbf{e}$ are matched, and $\mathbf{e}$ is the return letter corresponding to call $\mathbf{c}$ and vice versa. Formally, positions $|x| + 1$ and $|x| + |y| + 2$ in w are in the *matching relation*.

Visibly pushdown automata. A *(non-deterministic) visibly pushdown automaton* (**VPA**) $\mathcal{A}$ is a tuple $\mathcal{A} = (\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$ consisting of: (1) $\widehat{\Sigma}$, a finite pushdown alphabet, (2) Γ, a finite stack alphabet, (3) Q, a finite set of states, (4) $Q_0 \subseteq Q$, a set of initial states, (5) δ, a

transition relation defined as below, and (6) $F \subseteq Q$, a set of accepting states. The transition relation δ is the union of relations $\delta_c \cup \delta_l \cup \delta_r$, where $\delta_c \subseteq Q \times \Sigma_c \times Q \times \Gamma$, $\delta_l \subseteq Q \times \Sigma_l \times Q$, and $\delta_r \subseteq Q \times \Sigma_r \times \Gamma \cup \{\bot\} \times Q$.

Deterministic visibly pushdown automata. A **VPA** $(\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$ is deterministic (called **DVPA**), if $|Q_0| = 1$ and its transition relation δ is a function, which is the union of three functions: $\delta_c \colon Q \times \Sigma_c \to Q \times \Gamma$, $\delta_l \colon Q \times \Sigma_l \to Q$, $\delta_r \colon Q \times \Sigma_r \times \Gamma \cup \{\bot\} \to Q$.

Semantics of VPA. Consider a **VPA** $\mathcal{A} = (\widehat{\Sigma}, \Gamma, Q, Q_0, \delta, F)$. A *configuration* of $\mathcal{A}$ is a pair consisting of the current state $q \in Q$ of $\mathcal{A}$ and its current stack content $u \in \bot \cdot \Gamma^*$. A **VPA** defines an infinite-state transition system over its configurations $Q \times (\bot \cdot \Gamma^*)$, where configurations $Q_0 \times \{\bot\}$ are initial. For $a \in \widehat{\Sigma}$, let $\to^a_{\mathcal{A}}$ be an auxiliary relation defined as follows:

- $(q, u) \to^a_{\mathcal{A}} (q', uB)$, if a is a call letter and $(q, a, q', B) \in \delta_c$,
- $(q, u) \to^a_{\mathcal{A}} (q', u)$, if a is a local letter and $(q, a, q') \in \delta_l$,
- $(q, uB) \to^a_{\mathcal{A}} (q', u)$, if a is a return letter $(q, a, B, q') \in \delta_r$ and $B \in \Gamma$, and
- $(q, \bot) \to^a_{\mathcal{A}} (q', \bot)$, if a is a return letter $(q, a, \bot, q') \in \delta_r$.

For a word $w = a_1 \dots a_n$, we define $\to^w_{\mathcal{A}}$ as the composition of relations $\to^{a_1}_{\mathcal{A}}, \dots, \to^{a_n}_{\mathcal{A}}$, i.e., $(q, u_0) \to^w_{\mathcal{A}} (q', u_n)$ if there exist intermediate configurations $(q_1, u_1), \dots, (q_{n-1}, u_{n-1})$ such that for each $1 \leq i \leq n$ we have $(q_{i-1}, u_{i-1}) \to^{a_i}_{\mathcal{A}} (q_i, u_i)$. A word w is *accepted* by $\mathcal{A}$ if there are $q_0 \in Q_0$, $q \in F$ and $u \in \bot \cdot \Gamma^*$ such that $(q_0, \bot) \to^w_{\mathcal{A}} (q, u)$. The *language recognized by* $\mathcal{A}$, denoted $\mathcal{L}(\mathcal{A})$, is the set of words accepted by $\mathcal{A}$.

String rewriting. A *string rewriting system (SRS)* $\mathcal{R}$ over an alphabet Σ is a (possibly infinite) set of pairs of words (l, r) over Σ. A pair of words (l, r) from $\mathcal{R}$ is called a *string rewrite rule* and denoted with $l \to r$. For an SRS $\mathcal{R}$, we define a *single-step rewrite relation* $\to_{\mathcal{R}}$ over words from Σ^* as follows: for all $s, t \in \Sigma^*$, we have $s \to_{\mathcal{R}} t$ if and only if there are words $x, y \in \Sigma^*$ and a rewrite rule $(l, r) \in \mathcal{R}$ such that $s = xly$ and $t = xry$. The (string) rewriting relation $\to^*_{\mathcal{R}}$ is the transitive and reflexive closure of $\to_{\mathcal{R}}$. We will omit the $\mathcal{R}$ subscript if the SRS is clear from the context. For further details and notions on the topic of string rewriting see [16].

Active learning of languages and automata. The framework of active automata learning assumes an oracle, called a *Minimally Adequate Teacher*, which answers two types of queries about the target language $\mathcal{L}$:

- **membership queries**: given a $w \in \Sigma^*$, is $w \in \mathcal{L}$?, and
- **equivalence queries**: given an automaton $\mathcal{A}$, is $\mathcal{L}(\mathcal{A}) = \mathcal{L}$? If not, the teacher returns a counterexample, which is a word from the symmetric difference of $\mathcal{L}(\mathcal{A})$ and $\mathcal{L}$.

Additional types of queries are also possible [40]. The goal of the active learning algorithm is to return a minimal-size automaton recognizing the target language. The automaton model is connected to the type of the learned language: regular languages are represented by **DFA**, while **VPLs** are represented in active learning by **DVPA**, Single-Entry Visibly Pushdown Automata (SEVPA) or other subclasses of **DVPA** [2, 19]. Our method focuses on inferring answers to equivalence queries; hence, it does not depend on the specific active learning algorithm used, as long as it asks such queries.

3 Rewriting strings in visibly pushdown languages

We adapt an advice mechanism introduced for **DFA** in [24] to **DVPA**. The main aim of advice is to provide the learning algorithm with prior knowledge about the learned language, which is used to reduce the number of equivalence queries.

First, we present the general framework, which carries over from the regular case seamlessly. Next, We discuss which rewriting rules are compatible with **VPLs** and how to extend the SRS formalism to capture natural properties of **VPLs**.

3.1 Learning with advice

We adapt the framework of active learning with advice for **DFA** [24] to **DVPA**. The advice supplied to the learning algorithm is expressed via string rewriting systems (SRSs), which are connected to visibly pushdown languages through the following notions of consistency:

▶ **Definition 1** ([24]). *Let $\widehat{\Sigma}$ be a pushdown alphabet, $\mathcal{R}$ be an SRS over $\widehat{\Sigma}$ and $\mathcal{L}$ be a visibly pushdown language over $\widehat{\Sigma}$. We say that the SRS $\mathcal{R}$ is*

- *(fully) consistent with $\mathcal{L}$ if and only if for all words x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \in \mathcal{L} \Leftrightarrow y \in \mathcal{L}$.*
- *positively consistent with $\mathcal{L}$ if and only if for all x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \in \mathcal{L} \Rightarrow y \in \mathcal{L}$.*
- *negatively consistent with $\mathcal{L}$ if and only if for all x, y, if $x \to_{\mathcal{R}}^{*} y$, then $x \notin \mathcal{L} \implies y \notin \mathcal{L}$.*

Note that an SRS $\mathcal{R}$ is fully consistent with $\mathcal{L}$ if and only if it is both positively and negatively consistent.

With these notions of consistency in place, we can now present the active learning with advice framework.

Framework. The active **DVPA** learning with advice problem is defined as follows. The algorithm is given an *oracle* $\mathcal{O}$ answering membership and equivalence queries about the target visibly pushdown language $\mathcal{L}$ (and possibly other queries), and three *advice* SRSs:

- $\mathcal{R}_{=}$, which is consistent with $\mathcal{L}$, and
- $\mathcal{R}_{+}$, $\mathcal{R}_{-}$, which are respectively positively and negatively consistent with $\mathcal{L}$.

The algorithm's the task is to construct a **DVPA** recognizing $\mathcal{L}$. The advice is used to approximate equivalence checks, thereby reducing the number of equivalence queries posed to the oracle. Therefore, it can be integrated with any framework for active learning of visibly pushdown languages [8, 31] or **DVPA** [40] with possibly different types of queries provided that the oracle answers at least membership and equivalence queries. In particular, the oracle may also answer additional types of queries as in [40].

Using advice. We intercept equivalence queries posed by the learning algorithm and first check whether the advice SRS $\mathcal{R}$ is consistent (resp., positively consistent or negatively consistent) with the language of a given candidate automaton $\mathcal{A}$. We forward $\mathcal{A}$ to the oracle, only if it is consistent Otherwise, if it is inconsistent, without the oracle we conclude that $\mathcal{L}(\mathcal{A})$ differs from the target language with which $\mathcal{R}$ is consistent (resp., positively/negatively consistent). It suffices to compute a pair of words violating consistency (resp., positive/negative consistency), i.e., words x, y such that $x \to_{\mathcal{R}}^{*} y$ but exactly of x, y belongs to $\mathcal{L}$. Then, one of the words x, y must be a counterexample to $\mathcal{L}(\mathcal{A})$ being equal to the target language. Which one it is can be determined with a single membership query.

3.2 Rewrite rules compatible with VPLs

While the notion of consistency with the language is the same for **DVPA** as for **DFA**, the key difference lies in the choice of rewrite rules. With regular languages, any pair of words is a valid string rewrite rule, but words in visibly pushdown languages have an implicit structure induced by the matching relation between calls and returns. The intuition behind rewrite rules is that words are rewritten to other words that are equivalent with respect to a given language. Rewrite rules should therefore preserve the call–return matching outside the rewritten subword. In other words, rewriting should act locally and not affect the rest of

the word. While this is trivially true for regular words, it is not for pushdown words. For instance, by rewriting $c_1c_2c_3r_3r_2r_1$ with the rule $c_3r_3 \to r_3c_3$ we get a well-matched word with a different matching relation (color coded) $c_1c_2r_3c_3r_2r_1$.

▶ **Definition 2.** *A rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is matching-preserving over $\widehat{\Sigma}$ if and only if for all words $w, w' \in \widehat{\Sigma}^*$ such that $w = x\boldsymbol{\alpha}y$ and $w' = x\boldsymbol{\beta}y$, the matching relation between calls from x and returns from y in w coincides with the matching relation between calls from x and returns from y in w'. An SRS $\mathcal{R}$ is matching-preserving if all rules in $\mathcal{R}$ are matching-preserving.*

Matching-preserving rules do not break the matching between calls and returns that are not in the rewritten fragment of the word. A prominent subclass of matching-preserving rules is the set of *well-matched rules*, which arise naturally and enjoy more efficient algorithms for checking consistency:

▶ **Definition 3.** *A rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is well-matched over $\widehat{\Sigma}$ if and only if words $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ are well-matched. An SRS $\mathcal{R}$ is well-matched if all rules in $\mathcal{R}$ are well-matched.*

Every well-matched rule is matching-preserving. However, not every matching-preserving rule is well-matched: for instance, a rewrite rule $c_1c_2r_2 \to c_1$ is matching-preserving, but it is not well-matched. Most of our examples are well-matched rules.

3.3 Rewrite rules with variables

While we have restricted admissible rules to those that affect pushdown words locally, we face another problem. Natural properties of pushdown languages cannot be expressed with string rewriting rules, which we illustrate with the example below.

▶ **Example 4.** Consider the Dyck language $\mathcal{L}_2$ of well-matched brackets over two pairs of brackets: $\{\langle, \rangle\}$ and $\{[,]\}$. The language $\mathcal{L}_2$ satisfies the property that the order of $\langle$ and $[$ is irrelevant provided that each opening bracket is matched by its corresponding closing bracket. For instance, in every word substituting $\langle\,[\,]\,\rangle$ with $[\,\langle\,\rangle\,]$ neither breaks the matching between other brackets nor fixes unmatched brackets. Consequently, $\langle\,[\,]\,\rangle \to [\,\langle\,\rangle\,]$ rule is consistent with $\mathcal{L}_2$. Unfortunately, this property cannot be expressed by any finite set of string rewriting rules as it requires exchanging arbitrarily distant letters; for example, for any $n \geq 1$, the word $\langle\,\underbrace{[\,\ldots\,[}_{n}\,\langle\,\rangle\,\underbrace{]\,\ldots\,]}_{n}\,\rangle$ is equivalent to $\langle\,\langle\,\underbrace{[\,\ldots\,[}_{n}\,\underbrace{]\,\ldots\,]}_{n}\,\rangle\,\rangle$. This property can, however, be expressed using a word variable X ranging over well-matched words. Then, the rule

$$\langle\,[\,X\,]\,\rangle \to [\,\langle\,X\,\rangle\,]$$

expresses that the relative order of $\langle$ and $[$ is irrelevant provided their respective brackets match.

In order to express properties as illustrated in the above example, we extend rules with variables.

String rewriting systems with variables. Consider an infinite set of variables $\mathcal{X}$. A *string rewriting system with variables (**SRS-V**)* $\mathcal{R}[\mathcal{X}]$ is a finite set of rules $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ over $\widehat{\Sigma} \cup \mathcal{X}$ satisfying (1) every variable $X \in \mathcal{X}$ occurs at most once in $\boldsymbol{\alpha}$ and at most once in $\boldsymbol{\beta}$, and (2) every variable $X \in \mathcal{X}$ that occurs in $\boldsymbol{\beta}$, also occurs in $\boldsymbol{\alpha}$. We consider **SRS-Vs** as finite representations of infinite SRSs. That is, we say that a rule $\boldsymbol{\alpha} \to \boldsymbol{\beta} \in \mathcal{R}[\mathcal{X}]$

generates an infinite SRS $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ consisting of all ground instances obtained by substituting well-matched words for variables, with the same word substituted for each occurrence of a given variable across both $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$. For example, a rule $cXr \to X$ generates an infinite set $\{cr \to \epsilon, ccrr \to cr, \ldots\}$. Finally, an **SRS-V** $\mathcal{R}[\mathcal{X}]$ generates an SRS $[\![\mathcal{R}[\mathcal{X}]]\!]$ defined as the union of $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ over all rules $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ from $\mathcal{R}[\mathcal{X}]$.

▶ Remark 5. String rewriting rules can be considered as terms under the unary signature (all symbols have arity at most one) with a single variable. However, the variables in rules that we consider do not correspond to standard variables in the term-rewriting setting, but to context variables.

Properties of SRSs with variables. The notions defined for SRSs carry over to SRSs with variables by applying the definition to the generated SRS. That is, we say that a rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is

- consistent (resp., positively/negatively consistent) with a language $\mathcal{L}$ if and only if the generated SRS $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ is consistent (resp., positively/negatively consistent) with $\mathcal{L}$,
- matching-preserving (resp., well-matched) if and only if the generated SRS $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ is matching-preserving (resp., well-matched).

The listed properties extend to SRSs with variables, by requiring that every rule is consistent, well-matched, e.t.c. To check whether a rule with variables is matching-preserving (resp., well-matched) it suffices to delete variables, i.e., a rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is matching-preserving (resp. is well-matched) it holds if and only if the rule without variables obtained by deleting all variables from $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is matching-preserving (resp. is well-matched).

Well-matched instantiations. We restrict variables to range over well-matched words only, since non-well-matched instantiations need not be matching-preserving. Consider the rule $cXr \to X$ that is even well-matched and hence every rule in $[\![cXr \to X]\!]$ is well-matched. However, substituting X with rc yields $crcr \to rc$, which is not even matching-preserving, since $crcr$ is well-matched, while rc is not. Furthermore, as discussed in the following section, restricting to well-matched instantiations facilitates deciding consistency with a given **DVPA**.

4 Algorithms for checking consistency

We develop algorithms for deciding consistency (resp., positive or negative consistency). We discuss separately well-matched rules (Section 4.1) and the wider class of matching-preserving rules (Section 4.2). In both cases, we consider first rules without variables, and then the extension to rules with variables. Although the asymptotic complexity is the same in both cases, the algorithms for well-matched rules are conceptually simpler and admit more efficient implementations. Before studying particular types of rules, we make a general observation that reduces consistency checking to single-step rewriting.

Reduction to single-step rewriting. Observe that an **SRS-V** $\mathcal{R}$ is consistent with the language of $\mathcal{A}$ if and only if for every rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ and all words x, y, we have $x\boldsymbol{\alpha}y \in \mathcal{L}(\mathcal{A}) \iff x\boldsymbol{\beta}y \in \mathcal{L}(\mathcal{A})$. Indeed, if this property holds for every rule, then consistency under multi-step rewriting follows by transitivity. This holds due to the assumption that the $\mathcal{A}$ and $\mathcal{R}$ are over the same alphabet [24]. Analogously, for positive and negative consistency it suffices to verify the property for each individual rewriting step.

4.1 Consistency of well-matched rules

Ground well-matched rules. Consider a well-matched rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ without variables, i.e., a standard string rewriting rule. We extend the alphabet $\widehat{\Sigma}$ with a fresh local letter # and

construct two **DVPA** over the extended alphabet. Both **DVPA** extend $\mathcal{A}$ and differ only in their behavior on the letter #; in $\mathcal{A}_1$ for any state q, the transition over # leads to the state q' such that $\mathcal{A}$ moves from q to q' over $\boldsymbol{\alpha}$. Since $\boldsymbol{\alpha}$ is well-matched, the run over $\boldsymbol{\alpha}$ does not depend on the stack content and it does not modify the stack content. Thus, the effect of reading $\boldsymbol{\alpha}$ in the **DVPA** $\mathcal{A}$ is equivalent to a transition over a single local letter. The **DVPA** $\mathcal{A}_2$ is defined analogously, but # simulates the word $\boldsymbol{\beta}$ instead. Again, word $\boldsymbol{\beta}$ is well-matched. Therefore, to check consistency of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with $\mathcal{L}(\mathcal{A})$ it suffices to check whether there is a word w such that exactly one of **DVPA** $\mathcal{A}_1, \mathcal{A}_2$ accepts it, which can be done by a single reachability check on the product **DVPA** $\mathcal{A}_1 \times \mathcal{A}_2$, a **DVPA** simulating two copies of $\mathcal{A}$ in parallel. The visibility condition ensures that a single stack over the product of stack alphabets is sufficient to simulate both **DVPA**. Checking positive (resp., negative) consistency amounts to checking whether $\mathcal{L}(\mathcal{A}_1) \setminus \mathcal{L}(\mathcal{A}_2)$ (resp., $\mathcal{L}(\mathcal{A}_2) \setminus \mathcal{L}(\mathcal{A}_1)$ for negative consistency) is non-empty, which again amounts to a single reachability check.

▶ **Proposition 6.** *Consistency (resp., positive, negative consistency) of a ground well-matched rule with the language of a given **DVPA** $\mathcal{A}$ can be decided in polynomial time via a single reachability check on a **DVPA** obtained from the product $\mathcal{A} \times \mathcal{A}$.*

Reachability in pushdown automata can be checked in $\mathcal{O}(|\mathcal{A}|^3)$, with a slightly improved bound $\mathcal{O}(|\mathcal{A}|^3/\log(|\mathcal{A}|))$ for *recursive state machines* [18] both of which fall into $\widetilde{\mathcal{O}}(|\mathcal{A}|^3)$. Due to the lack of better upper bounds [18, 7], the complexity of reachability in pushdown systems is often referred to as the "cubic bottleneck." Consequently, reachability in the product $\mathcal{A} \times \mathcal{A}$ amounts to $\widetilde{\mathcal{O}}(|\mathcal{A}|^6)$. However, checking consistency of an SRS with the language of a **DFA** can be performed locally on the structure of the **DFA** without checking reachability [24]. In contrast, the following example illustrates that for pushdown systems, checking consistency must involve reachability.

▶ **Example 7.** Consider a **DVPA** $\mathcal{B}$ depicted in Figure 1, which is over the pushdown alphabet $(\{c\}, \{l_1, l_2\}, \{r\})$ such that local letters l_1, l_2 induce self-loops on all states except for q_0. Its stack alphabet is a singleton $\{a\}$. In the state q_0 the **DVPA** moves over l_1 to q_1 and over l_2 to q_2. The automaton behaves differently in states q_1, q_2 if the stack is non-empty, while these states are indistinguishable if the stack is empty. Finally, q_0 is reachable only with non-empty stack, i.e., q_0 is reachable, but the configuration $(q_0, \bot)$ is not. The rule $l_1 \to l_2$ is consistent with $\mathcal{L}(\mathcal{B})$, but establishing this requires reachability analysis.

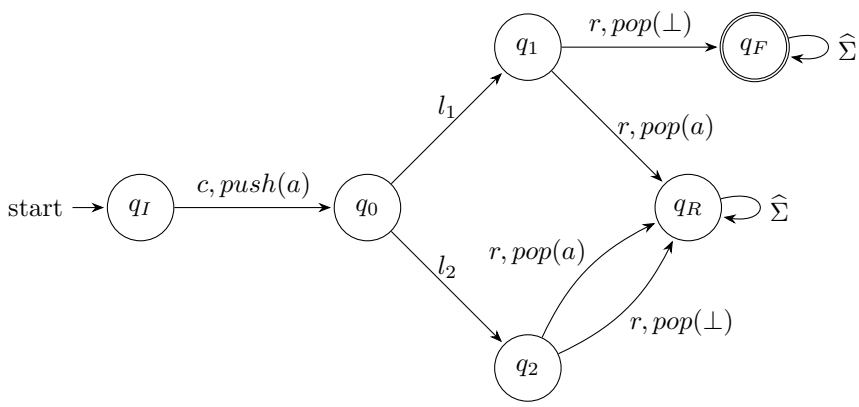


Figure 1 The automaton $\mathcal{B}$. All transitions other than depicted are self-loops

Non-ground well-matched rules. Consider a well-matched rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with variables. The general approach, as in the ground case, is to reduce consistency checking to reachability in a product **VPA**. We construct a **VPA** $\mathcal{A}_{\text{con}}$ that simulates two copies of $\mathcal{A}$ over the alphabet extended with a fresh local letter $\#$. The **VPA** $\mathcal{A}_{\text{con}}$ is built on the product $\mathcal{A} \times \mathcal{A}$, whose stack track contents of both copies of $\mathcal{A}$. The fresh local letter $\#$ simulates the word $\boldsymbol{\alpha}$ in the first copy and the word $\boldsymbol{\beta}$ in the second copy. However, there are two key differences. First, since $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ can be instantiated to (infinitely many) different words, transitions over $\#$ are non-deterministic. Second, the transitions in each copy of $\mathcal{A}$ cannot be defined independently; they must be defined jointly on the product to ensure that both sides of the rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ are instantiated to the same word. For example, for a rule $cXr \to X$, there may be a transition in $\mathcal{A}$ from q to q_1 over cw_1r and from q to q_2 over w_2, but we must ensure that $w_1 = w_2$. To this end, we compute a set $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ of triples (q, s_1, s_2) such that there are words α, β satisfying: (1) $s_1 \neq s_2$, (3) $(q, u) \to^{\alpha}_{\mathcal{A}} (s_1, u)$ and $(q, u) \to^{\beta}_{\mathcal{A}} (s_2, u)$ for every stack content u, and (4) $\alpha \to \beta$ is an instance of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ obtained by substituting well-matched words for variables. Using $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, we define a **VPA** $\mathcal{A}_{\text{con}}$ over the pushdown alphabet $\widehat{\Sigma}$ extended with a local letter $\#$ as an extension of $\mathcal{A} \times \mathcal{A}$ such that for a state (q_1, q_2) of $\mathcal{A}_{\text{con}}$: (a) if $q_1 \neq q_2$, then $\mathcal{A}_{\text{con}}$ has no transition over $\#$, and (b) if $q_1 = q_2$, then for every $(q_1, s_1, s_2) \in \Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, the tuple $\langle (q_1, q_1), \#, (s_1, s_2) \rangle$ is a transition in $\mathcal{A}_{\text{con}}$. Intuitively, the automaton only applies the rule when both copies are in the same state, and hence words with more than one $\#$ letter are rejected. Finally, the automaton accepts if in the reached state (q_1, q_2), exactly one of q_1, q_2 is accepting in $\mathcal{A}$. Observe that the **VPA** $\mathcal{A}_{\text{con}}$ accepts exactly words w that are witnesses of inconsistency of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ with $\mathcal{L}(\mathcal{A})$, i.e., any accepted word w contains exactly one occurrence of $\#$. Furthermore, there is a ground instance $\alpha \to \beta$ of the rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ obtained by substituting well-matched word for variables, and by substituting $\#$ with α (resp., β) yields w_1 (resp., w_2) satisfying (i) w_1 can be rewritten on one step to w_2 with the rule $\alpha \to \beta$, and (ii) exactly one of w_1, w_2 belongs to $\mathcal{L}(\mathcal{A})$. By modifying the acceptance condition, we can also decide positive and negative consistency.

In the following, we focus on computing $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ for rules with variables. It suffices to compute how well-matched words transform pairs of states of the **DVPA**.

Computing $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$. We compute a quaternary relation $\mathcal{B}_{(4)} \subseteq Q^4$ such that $(q_1, q_2, s_1, s_2) \in \mathcal{B}_{(4)}$ if and only if there is a well-matched word w such that $q_1 \to^{w}_{\mathcal{A}} s_1$ and $q_2 \to^{w}_{\mathcal{A}} s_2$. It is computed iteratively in a similar way to reachability in pushdown automata, but applied to the product automaton $\mathcal{A} \times \mathcal{A}$.

▶ **Lemma 8.** *Given a **DVPA** $\mathcal{A}$, the relation $\mathcal{B}_{(4)}$ can be computed in polynomial time in the size of $\mathcal{A}$.*

We extend the alphabet $\widehat{\Sigma}$ to $\widehat{\Sigma}_+$ by adding a pair of fresh local letters x^L and x^R for every variable X occurring in $\boldsymbol{\alpha} \to \boldsymbol{\beta}$. Next, we substitute each variable X with its corresponding local letters: x^L for X in $\boldsymbol{\alpha}$ and x^R for X in $\boldsymbol{\beta}$, obtaining words $\bar{\alpha}$ and $\bar{\beta}$ over the extended alphabet. Then, we iterate over all possible assignments from variables of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ to $\mathcal{B}_{(4)}$. For a given assignment σ, we extend the **DVPA** $\mathcal{A}$ to a **VPA** $\mathcal{A}_\sigma$ working over the extended alphabet, where transitions over new letters are defined as follows. For every variable X, if $\sigma(X) = (q_1, q_2, s_1, s_2)$, then $\mathcal{A}_\sigma$ has a transition over x^L from q_1 to s_1 and over x^R from q_2 to s_2; there are no other transitions over x^L or x^R. Finally, for every triple (q, s_1, s_2), if $(q, \bot) \to^{\bar{\alpha}}_{\mathcal{A}_\sigma} (s_1, \bot)$ and $(q, \bot) \to^{\bar{\beta}}_{\mathcal{A}_\sigma} (s_2, \bot)$, we add the triple (q, s_1, s_2) to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$. Note that the stack content $\bot$ does not influence reachability since $\bar{\alpha}, \bar{\beta}$ are well-matched.

▶ **Lemma 9.** *Deciding membership in $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ over well-matched rules is in NP. Under a fixed bound on the number of variables per rule, it is decidable in polynomial time.*

In consequence, checking inconsistency can be done in NP, which gives coNPtest for consistency:

▶ **Theorem 10.** *Consistency (resp., positive, negative consistency) of a well-matched rule with the language of a given* ***DVPA*** $\mathcal{A}$ *can be decided in coNP. Under a fixed bound on the number of variables per rule, it is decidable in polynomial time.*

Complexity of checking (positive) consistency. The proposed algorithm checks all assignments of transitions for variables, which is exponential in the number of variables; it works in polynomial time if the number of variables is bounded. Significantly improving upon this complexity is unlikely as the membership problem for $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ is NP-complete in general, and deciding positive (resp. negative) consistency is coNP-complete. The hardness result holds already for **DFA**, which can be considered as **DVPA** over only local letters.

▶ **Theorem 11.** *(1) The problem of deciding, given a rewrite rule* $\boldsymbol{\alpha} \to \boldsymbol{\beta}$*, a DFA* $\mathcal{A}$ *and its states* q, s_1, s_2*, whether* (q, s_1, s_2) *belongs to* $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$*, is NP-complete. (2) The positive (resp. negative) consistency problem over well-matched rules against a regular language given by a* ***DFA*** *is coNP-complete.*

Proof's sketch. We show thus by reduction from the 3-SAT problem. Given a formula φ, we rename variables to obtain φ^*, in which each variable occurs at most once. Then, $\mathcal{A}$ moves over some instance of $\boldsymbol{\alpha}$ from q to s_1 if the variable substitution of $\boldsymbol{\alpha}$ encodes a truth assignment, where all copies of the same variable have the same value. Next, $\mathcal{A}$ moves over some instance of $\boldsymbol{\beta}$ from q to s_2 if the variable substitution of $\boldsymbol{\beta}$ encodes a truth assignment satisfying the formula φ. Thus, (q, s_1, s_2) belongs to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ if and only if φ is satisfiable. Furthermore, we can reduce unsatisfiablity of φ to positive (resp. negative) consistency. We ensure that whenever a truth assignment is consistent among copies of the same propositional variable, it does not satisfy φ^*. Thus, φ is unsatisfiable. ◀

The coNP-hardness result for positive (resp. negative) consistency does not extend to consistency, and hence the complexity lower bound for checking consistency of well-matched rules remains open.

4.2 Consistency of matching-preserving rules

We turn to matching-preserving rules that are not well-matched. The overall approach is similar to the well-matched case; we extend the alphabet with auxiliary letters and consider the product $\mathcal{A} \times \mathcal{A}$. The key difference is that in well-matched rules, both sides are stack-independent and stack-neutral, whereas matching-preserving rules may depend on and modify the stack. As a result, instead of local letters, we extend the alphabet with call and return letters.

Ground matching-preserving rules. Consider a matching-preserving rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ without variables that is not well-matched. Since the rule is matching-preserving, $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ have the same number m of unmatched calls and the same number k of unmatched returns; since it is not well-matched, there are unmatched letters (at least one of k, m is non-zero); assume that $k > 0$ (the case $k = 0$ and $m > 0$ is similar). Furthermore, $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ can be partitioned into well-matched words and unmatched calls and returns, where unmatched returns precede unmatched calls. It follows that

$$\boldsymbol{\alpha} = v_0^L \cdot r_1^L \cdot v_1^L \ldots r_k^L \cdot v_k^L \cdot c_1^L \cdot v_{k+1}^L \cdot c_2^L \ldots c_m^L \cdot v_{k+m}^L$$
$$\boldsymbol{\beta} = v_0^R \cdot r_1^R \cdot v_1^R \ldots r_k^R \cdot v_k^R \cdot c_1^R \cdot v_{k+1}^R \cdot c_2^R \ldots c_m^R \cdot v_{k+m}^R$$

where v_i^L, v_i^R are well-matched words, r_i^L, r_i^R are return letters and c_i^L, c_i^R are call letters. As in the well-matched case, let $\mathcal{A}_E$ be an extension of the product automaton $\mathcal{A} \times \mathcal{A}$, i.e., its set of states is $Q \times Q$ and its stack alphabet is $\Gamma \times \Gamma$. The automaton $\mathcal{A}_E$ works over an extended alphabet, where the additional letters are k fresh return letters $\overline{\mathbf{r}}_1, \ldots, \overline{\mathbf{r}}_k$ and m fresh call letters $\overline{\mathbf{c}}_1, \ldots, \overline{\mathbf{c}}_m$. The transition over $\overline{\mathbf{r}}_1$ simultaneously simulates reading $v_0^L r_1^L v_1^L$ on the first component of $\mathcal{A}_E$ and $v_0^R r_1^R v_1^R$ on the second. In doing so, it pops one symbol from the product stack, which encodes a pair of stack symbols from the two copies of $\mathcal{A}$. Formally, $\delta_E((q_1, q_2), \overline{\mathbf{r}}_i, (b_1, b_2)) = (s_1, s_2)$ if $(q_1, ub_1) \rightarrow_{\mathcal{A}}^{v_0^L r_1^L v_1^L} (s_1, u)$ and $(q_2, ub_2) \rightarrow_{\mathcal{A}}^{v_0^R r_1^R v_1^R} (s_2, u)$ for every stack content u. For the remaining return letters $\overline{\mathbf{r}}_i$, where $i \in \{2, \ldots, k\}$, the transitions on $\overline{\mathbf{r}}_i$ simulate reading $r_i^L v_i^L$ on the first component and $r_i^R v_i^R$ on the second component. For each new call letter $\overline{\mathbf{c}}_i$, the transition over $\overline{\mathbf{c}}_i$ simultaneously simulates reading $c_i^L v_{k+i}^L$ on the first component and $c_i^R v_{i+k}^R$ on the second. Finally, a state (q_1, q_2) of $\mathcal{A}_E$ is accepting if exactly one of q_1, q_2 is accepting in $\mathcal{A}$.

The following condition holds:

▷ Claim 12. A word $w_1 \overline{\mathbf{r}}_1 \ldots \overline{\mathbf{r}}_k \overline{\mathbf{c}}_1 \cdots \overline{\mathbf{c}}_m w_2$, where w_1, w_2 are over the original alphabet $\widehat{\Sigma}$, is accepted by $\mathcal{A}_E$ if and only if $\mathcal{A}$ accepts exactly one of the words $w_1 \boldsymbol{\alpha} w_2, w_1 \boldsymbol{\beta} w_2$, which holds precisely when $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$ is inconsistent with $\mathcal{L}(\mathcal{A})$.

Therefore, the rule $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$ is consistent with $\mathcal{L}(\mathcal{A})$ if and only if the language $\mathcal{L}(\mathcal{A}_E) \cap \mathcal{L}(\widehat{\Sigma}^* \overline{\mathbf{r}}_1 \ldots \overline{\mathbf{r}}_k \overline{\mathbf{c}}_1 \cdots \overline{\mathbf{c}}_m \widehat{\Sigma}^*)$ is empty.

By changing the accepting states of $\mathcal{A}_E$ as discussed in the well-matched case, we can decide positive and negative consistency using emptiness of $\mathcal{L}(\mathcal{A}_E) \cap \mathcal{L}(\widehat{\Sigma}^* \overline{\mathbf{r}}_1 \ldots \overline{\mathbf{r}}_k \overline{\mathbf{c}}_1 \cdots \overline{\mathbf{c}}_m \widehat{\Sigma}^*)$. In consequence, we have:

▶ **Proposition 13.** *Consistency (resp., positive, negative consistency) of a ground matching-preserving rule with the language of a given* ***DVPA*** $\mathcal{A}$ *can be decided in polynomial time via a single reachability check on a* ***DVPA*** *obtained from the product* $\mathcal{A} \times \mathcal{A} \times \mathcal{B}$*, where* $\mathcal{B}$ *is a* ***DFA*** *depending only on the rule.*

Now, we can move to rules with variables. We only briefly comment on the extension as it is similar as for well-matched rules.

Matching-preserving rules with variables. Variables in matching-preserving rules can be eliminated as in the case of well-matched rules. Consider a matching-preserving rule $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$. It suffices to extend the alphabet with local letters x^L, x^R for each variable and then consider a ground rule obtained from $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$ by replacing the occurrence of every variable X with x^L in $\boldsymbol{\alpha}$ and with x^R in $\boldsymbol{\beta}$ whenever appears. Finally, as described in Section 4.1 for the well-matched case, we iterate over all possible assignments from variables of $\boldsymbol{\alpha} \rightarrow \boldsymbol{\beta}$ to $\mathcal{B}_{(4)}$. In consequence, we have the following:

▶ **Theorem 14.** *Consistency (resp., positive, negative consistency) of a matching-preserving rule with the language of a given* ***DVPA*** $\mathcal{A}$ *can be decided in* coNP*. Under a fixed bound on the number of variables per rule, it is decidable in polynomial time.*

5 Rule synthesis

In the previous section, we have studied how advice provided via an SRS reduces the number of equivalence queries necessary to learn an unknown language. In this section, we address the converse: *synthesis* of rewrite rules consistent with a given visibly pushdown language. Synthesized rewrite rules can serve as an explanation of the visibly pushdown language under consideration, capturing its key properties through concise rewrite rules.

There are two possible variants of this problem:

Synthesis from Automata: Given a **DVPA**, generate string rewrite rules consistent with the language of the given **DVPA**.

Synthesis from Oracles: Given an oracle for an unknown visibly pushdown language, generate string rewrite rules consistent with the language.

While *Synthesis from Oracles* seems more natural, we show that even given a **DVPA**, the synthesis problem is challenging for two reasons: high computational complexity and the lack of a clear objective — it is not obvious what constitutes a "relevant" rule. Therefore, *Synthesis from Oracles* suffers from these same shortcomings, and further progress in *Synthesis from Automata* is required before *Synthesis from Oracles* can be effectively addressed. Consequently, for the remainder of this section we focus on *Synthesis from Automata*.

We consider a **DVPA** $\mathcal{A}$ with the set of states Q over a non-trivial alphabet, i.e., either there are at least two local letters or at least one pair of a call and a return letter. Under these assumptions, there are exponentially many well-matched words.

We establish a polynomial upper bound on the lengths of words in minimal-length rules consistent with $\mathcal{A}$, which implies that such rules can be computed in non-deterministic polynomial time. In order to achieve that, we give sufficient conditions for pairs of words to form a rewrite rule consistent with $\mathcal{L}(\mathcal{A})$. We consider first rules without variables, and then move to rules with a single variable, which is finally generalized to any bounded number of variables.

The length of a minimal consistent well-matched rule: the ground case. Consider a well-matched word w. We associate with this word a function $f_w \colon Q \to Q$. Such functions correspond to equivalence classes of the syntactic congruence of a **DFA**. Formally, for a state $q \in Q$ we define $f_w(q) = q'$ if and only if for some stack content u, we have $(q, u) \to^a_{\mathcal{A}} (q', u)$. Note that the existential quantifier over u can be replaced by a universal one, since transitions over well-matched words are stack-independent. If w_1, w_2 are well-matched words such that $f_{w_1} = f_{w_2}$, then for all words x, y we have $xw_1y \in \mathcal{L}(\mathcal{A}) \iff xw_2y \in \mathcal{L}(\mathcal{A})$ [2, Theorem 1]. It follows that $w_1 \to w_2$ is a string rewrite rule consistent with $\mathcal{L}(\mathcal{A})$. Setting $n = |Q|$, there are n^n functions from Q to Q, and hence there are $n^n = 2^{\mathcal{O}(n \log n)}$ distinct functions f_w over all well-matched words w. There are $2^{\Theta(k)}$ well-matched words of the length k, and hence among well-matched words of length $\mathcal{O}(n \log n)$ there is a pair of well-matched words $\boldsymbol{\alpha}, \boldsymbol{\beta}$ such that $f_{\boldsymbol{\alpha}} = f_{\boldsymbol{\beta}}$. It follows that $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is a ground well-matched rule consistent with $\mathcal{L}(\mathcal{A})$.

The length of a minimal consistent well-matched rule: the general case. For rules with variables, we consider only a subset of rules and only a sufficient condition for consistency. Nevertheless, we obtain asymptotically the same bound. Consider a rewrite rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ of the form:

$$w_0 X_1 w_1 X_2 \dots X_k w_k \to v_0 X_1 v_1 X_2 \dots X_k v_k$$

where the order of variables in $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ is the same and all words w_i, v_i are well-matched words over $\widehat{\Sigma}$, i.e., do not contain variables. Observe that if the corresponding words w_i and v_i define the same function, i.e., $f_{w_0} = f_{v_0}, \dots, f_{w_k} = f_{v_k}$, then the rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is consistent with the language of $\mathcal{A}$. To see this, consider an instantiation of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$, i.e., consider a sequence of well-matched words $b_1, \dots, b_k$, which we substitute for $X_1, \dots, X_k$ obtaining the ground rewrite rule:

$$w_0 b_1 w_1 b_2 \dots b_k w_k \to v_0 b_1 v_1 b_2 \dots b_k v_k$$

Observe that the left hand side and the right hand side define the same function, and hence this ground rule is consistent with $\mathcal{L}(\mathcal{A})$. It follows that every rule from $[\![\boldsymbol{\alpha} \to \boldsymbol{\beta}]\!]$ is consistent with $\mathcal{L}(\mathcal{A})$, which by definition means that $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is consistent with $\mathcal{L}(\mathcal{A})$. Now, setting $n = |Q|$, there are $n^{k \cdot n}$ different combinations of k functions from Q to Q. The number of well-matched words is $2^{\Omega(n)}$ and hence there is a constant $c > 0$ such that among words of the form $w_0 X_1 w_1 X_2 \dots X_k w_k$ such that the sum of lengths of $w_0, \dots, w_k$ is bounded by $c \cdot k \cdot n \cdot \log(n)$, there is a pair of words that forms a rewriting rule consistent with $\mathcal{L}(\mathcal{A})$.

Rule synthesis in non-deterministic polynomial time. The counting argument shows that for a given **DVPA** $\mathcal{A}$ there are short (of polynomial length in $|\mathcal{A}|$) rewrite rules that are consistent with $\mathcal{L}(\mathcal{A})$. Since testing whether a given rule (with a bounded number of variables) is consistent with $\mathcal{L}(\mathcal{A})$ is decidable in polynomial time, we can find (some) rules consistent with $\mathcal{L}(\mathcal{A})$ in non-deterministic polynomial time.

Rules of exponential length. While minimal-length rules are of polynomial length, some consistent rules are of exponential length even if we consider "nonredundant" rules. Even in the **DFA** case, consider automata $\mathcal{A}_n$ such that $|Q| = n$ and for all $a \in \Sigma$, the function f_a is the same permutation of states of an exponential order, e.g., the composition of disjoint cycles of prime orders $p_1, \dots, p_k$ has the order $p_1 \cdot \ldots \cdot p_k$. Then, there exist words $w \neq \epsilon$ such that $w \to \epsilon$ is a rule consistent with $\mathcal{L}(\mathcal{A}_n)$, but every such word w has length at least exponential in n.

While we have shown that rules can be generated in non-deterministic polynomial time, a natural question is whether this is optimal.

Complexity of finding consistent rules. We have established an upper bound on the minimal length of words defining the same function. We show that computing the exact minimal length of such words is difficult, which implies that finding a minimal-length rewriting rule, which is consistent with a given **DVPA** is difficult as well. Furthermore, the problem is already difficult for **DFA**.

We consider the *minimal collision* problem defined as follows: given a **DFA** $\mathcal{A}$ and $k > 0$, does there exist a pair of different words w_1, w_2 such that (1) $|w_1|, |w_2| \leq k$, and (2) $f_{w_1} = f_{w_2}$. As we have discussed above for the ground rewrite rules, if $|\Sigma| \geq 2$, then for every $k > |\mathcal{A}| \cdot \log(|\mathcal{A}|)$ the answer is yes. Consequently, the problem is in NP. We show that it is NP-complete.

▶ **Lemma 15.** *The minimal collision problem is NP-complete.*

Proof's sketch. The proof is by reduction from the 3-SAT problem. The main difficulty of the proof is to ensure that if the formula is unsatisfiable, then no pair of short words defines the same function from Q to Q . We construct a gadget **DFA** such that unless one of the designed letters occurs in a word, each word of length at most n generates a unique function. We consider the alphabet consisting of literals from a given 3-CNF formula. With that gadget **DFA**, we enforce that short words having a collision must contain a variable or its negation, i.e., represent an assignment, and these words must contain a literal from each clause. ◀

In the minimal collision problem, both sides of the rule are unknown words. We discuss another decision problem associated with rule synthesis, in which the right-hand side of the rule is fixed. For simplicity, we consider the right-hand side to be a local letter. The *bounded rewriting to single letter* problem asks, given a **DFA** $\mathcal{A}$ over Σ, a letter $a \in \Sigma$, and a bound $k > 0$, whether there exists a word $w \in (\Sigma \setminus \{a\})^*$ such that (1) $w \to a$ is a rule consistent with $\mathcal{L}(\mathcal{A})$, and (2) $|w| \leq k$. This problem is closely related to the *membership* problem for monoids [34] and groups [30]. The membership problem for monoids reduces to

the *bounded automata intersection* problem [34, 49], which asks, given $m, k > 0$ and **DFA** $\mathcal{A}_1, \ldots, \mathcal{A}_m$, whether there exists a word w such that $|w| \leq k$ and $w \in \mathcal{L}(\mathcal{A}_i)$ for every i. The bounded automata intersection problem is PSPACE-complete if k is given in binary [34] and NP-complete if k is given in unary [49]. The same complexity bounds apply to the bounded rewriting to single letter problem as there is a straightforward reduction from the bounded automata intersection problem. Clearly, the problem is in PSPACE for k given in binary and NP for k given in unary.

▶ **Lemma 16.** *The bounded rewriting to a single letter with the parameter k is (a) PSPACE-complete, if k is given in binary, and (b) NP-complete, if k is given in unary.*

In summary, there is no algorithm for generating minimal-length consistent rules in deterministic polynomial time unless P= NP.

6 Empirical evaluation

We evaluate our framework by expressing natural VPL properties as rewrite rules with variables. Our experiments confirm that this learning with advice approach yields a substantial reduction in equivalence queries, confirming the practical advantages of the proposed method.

6.1 Examples of rewrite rules

We consider the following types rules:

- *Well-matched commutativity*: $c_1c_2Xr_2r_1 \to c_2c_1Xr_1r_2$.
- *Well-matched join*: $cXrcYr \to cXYr$.
- *Well-matched cancellation*: $cXr \to \epsilon$.

  c, r are comment tags
- *Well-matched idempotency*: $ccXrr \to cXr$.

6.2 Implementation

Baseline. As a baseline for our experiments, we have implemented a polynomial-time active learning algorithm for **DVPA** [40]. The algorithm from [40] assumes a stronger teacher model than the typical MAT; it allows queries extended membership queries that read as well modify the stack content and hence it depends on a concrete **DVPA** rather than **VPL**. On the positive side, the learning algorithm works in polynomial time.

The advice mechanism. We have extended the baseline algorithm with the advice mechanism, which intercepts equivalence queries before passing them to the the equivalence oracle performs the consistency check. The advice is given via rewrite rules with a single variable, and to make it more efficient instead of the exact algorithm presented in the paper, we substitute for the variable one of few selected well-matched words.

I have tested up to 200 well-matched words for a single check

A single consistency check is performed for all ground rules constructed in this way. We have observed that often testing with the substitution $X \mapsto \epsilon$ suffices to detect inconsistency.

Is consistency test for ground rules optimized?

Equivalence oracle. We have implemented two types of equivalence test: the exact test based on reachability in the product automaton, and the probabilistic test based on testing conformance of automata on a large number (10^5 and 10^6) of randomly generated words.

In the probabilistic test faster? – it's much faster

Are there any other important details? – the length of random words was restricted to 100

6.3 Experiments

We have conducted a series of experiments to address the following research questions:

Q1 What is the average reduction in the number of equivalence queries avoided using advice? Are there any features that influence performance of the advice mechanism?

Q2 Does the use of the advice mechanism lead to better running time of the whole learning setup (i.e., the runtime of the learning algorithm plus the runtime of the probabilistic equivalence oracle)?

Q3 Does the use of the advice mechanism lead to higher reliability of the learning setup in presence of the probabilistic equivalence oracle?

Benchmarks. We have generated random benchmark **DVPA** with multiple parameters such as the size of the alphabet (the number of call, local and return letters are specified separately), the size of the stack alphabet, number of states, density of transitions (the ratio of defined transitions to all possible transitions, where undefined ones are treated as valid transitions to a special sink state

Is 'density of transitions' worth mentioning?

) and the ratio of accepting states among all states. We have observe that the number of states has the main infulence on the number of equivalence queries, while the size of the alphabet or the size of the stack alphabet have negligable impact on the number of required equivalence queries. We have also implemented random generation of a **DVPA** that is (a) a Single-Entry VPA (SEVPA), (b) Multi-Entry VPA (MeVPA), (c) Call-Driven Automata (CDA), and Expanded Call-Driven Automata (eCDA). Moreover, we have implemented random generation of **DVPA** that are consistent which each type of advice discussed above:

- For *well-matched commutativity*, we generate two **DVPA** over disjoint alphabets, take the product of these automata, where undefined transitions are defined as self-loops.
- For *well-matched cancellation*, we generate a **DVPA** $\mathcal{A}$ randomly over the alphabet without letter c, r. Then, $\mathcal{A}$ is extended to $\mathcal{A}'$ over the alphabet including c, r such that the rule $cXr \to \epsilon$ is consistent.
- For *Well-matched idempotency*, we generate a SEVPA with the condition that returning to the same module does not change the state, which implies that $ccXrr \to cXr$ is consistent.

We present the results of two sets of experiments. The first set is based on an exact equivalence check and evaluates the reduction in the number of equivalence queries as well as the total execution time. The second set is based on a probabilistic equivalence check and evaluates the reliability and the execution time.

Exact equivalence check.

Probabilistic equivalence check. In this section, we evaluate two aspects: the impact of SRS on reliability and the reduction in processing time. Since SRS enables performing equivalence testing in two stages, we compare it against a baseline without SRS using two configurations: 10^5 and $2 \cdot 10^5$ random words. The second configuration is intended to verify whether the improvements observed with SRS are not merely a consequence of a larger number of random tests.

The experimental setup consists of 10^4 randomly generated automata, each with 10 to 25 states. The input alphabet was generated individually for each automaton, with slight variations depending on the considered property. In particular, the number of symbols of each type ranged from 2 to 5, while the number of stack symbols ranged from 2 to 16. The number of well-matched words used for SRS was limited to 200.

SRS rule	Metric	10^5 random words	$2 \cdot 10^5$ random words	10^5 with SRS
Cancellation	Success ratio	30.29%	...	57.19%
	Avg. time [s]	0.69	...	2.6
Commutativity	Success ratio	73.93%	...	74.11%
	Avg. time [s]	0.69	...	2.0
Idempotency	Success ratio	91.0%	...	91.69%
	Avg. time [s]	0.38	...	0.92

Table 1 Comparison of success ratio and average execution time for different SRS rule types

Summary.

References

1. Andreas Abel and Jan Reineke. Gray-box learning of serial compositions of mealy machines. In *NFM 2016*, pages 272–287, 2016.
2. Rajeev Alur, Viraj Kumar, Parthasarathy Madhusudan, and Mahesh Viswanathan. Congruences for visibly pushdown languages. In *ICALP 2005*, pages 1102–1114. Springer, 2005.
3. Rajeev Alur and P. Madhusudan. Visibly pushdown languages. In László Babai, editor, *Proceedings of the 36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004*, pages 202–211. ACM, 2004.
4. Dana Angluin. Learning k-bounded context-free grammars. *Research report. Yale University. Dept. of Computer Science*, 557, 1987.
5. Dana Angluin. Learning regular sets from queries and counterexamples. *Information and computation*, 75(2):87–106, 1987.
6. Dana Angluin and Michael Kharitonov. When won't membership queries help? *Journal of Computer and System Sciences*, 50(2):336–355, 1995.
7. A. R. Balasubramanian, Dmitry Chistikov, and Rupak Majumdar. Pushdown model checking above the cubic bottleneck. In *40th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2025, Singapore, June 23-26, 2025*, pages 622–635. IEEE, 2025.
8. Benoît Barbot, Benedikt Bollig, Alain Finkel, Serge Haddad, Igor Khmelnitsky, Martin Leucker, Daniel Neider, Rajarshi Roy, and Lina Ye. Extracting context-free grammars from recurrent neural networks using tree-automata learning and a* search. In Jane Chandlee, Rémi Eyraud, Jeff Heinz, Adam Jardine, and Menno van Zaanen, editors, *Proceedings of the 15th International Conference on Grammatical Inference, 23-27 August 2021, Virtual Event*, Proceedings of Machine Learning Research, pages 113–129. PMLR, 2021.
9. Osbert Bastani, Rahul Sharma, Alex Aiken, and Percy Liang. Synthesizing program input grammars. In Albert Cohen and Martin T. Vechev, editors, *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, pages 95–110. ACM, 2017.
10. Amos Beimel, Francesco Bergadano, Nader Bshouty, Eyal Kushilevitz, and Stefano Varricchio. Learning functions represented as multiplicity automata. *Journal of the ACM*, 47, 10 1999.
11. Bachir Bendrissou, Rahul Gopinath, and Andreas Zeller. "synthesizing input grammars": a replication study. In Ranjit Jhala and Isil Dillig, editors, *PLDI '22: 43rd ACM SIGPLAN*

International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022, pages 260–268. ACM, 2022.

12 Raphaël Berthon, Adrien Boiret, Guillermo A. Pérez, and Jean-François Raskin. Active learning of sequential transducers with side information about the domain. In *DLT 2021*, pages 54–65, 2021.

13 Satwik Bhattamishra, Michael Hahn, and Varun Kanade. Automata learning and identification of the support of language models. In *International Conference on Learning Representations ICLR 2026*. OpenReview.net, 2026.

14 Henrik Björklund, Johanna Björklund, and Petter Ericson. On the regularity and learnability of ordered DAG languages. In Arnaud Carayol and Cyril Nicaud, editors, *CIAA 2017*, volume 10329 of *Lecture Notes in Computer Science*, pages 27–39. Springer, 2017.

15 Benedikt Bollig, Peter Habermehl, Carsten Kern, and Martin Leucker. Angluin-style learning of NFA. In *IJCAI 2009*, pages 1004–1009, 2009.

16 Ronald V. Book and Friedrich Otto. *String-Rewriting Systems*. Texts and Monographs in Computer Science. Springer, 1993.

17 Véronique Bruyère, Guillermo A. Pérez, and Gaëtan Staquet. Validating streaming JSON documents with learned vpas. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *TACAS 2023*, Lecture Notes in Computer Science, pages 271–289. Springer, 2023.

18 Swarat Chaudhuri. Subcubic algorithms for recursive state machines. In George C. Necula and Philip Wadler, editors, *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 159–169. ACM, 2008.

19 Patrick Chervet and Igor Walukiewicz. Minimizing variants of visibly pushdown automata. In Ludek Kucera and Antonín Kucera, editors, *Mathematical Foundations of Computer Science 2007, 32nd International Symposium, MFCS 2007, Ceský Krumlov, Czech Republic, August 26-31, 2007, Proceedings*, Lecture Notes in Computer Science, pages 135–146. Springer, 2007.

20 Colin de la Higuera. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, New York, NY, USA, 2010.

21 Frank Drewes and Johanna Högberg. Learning a regular tree language from a teacher. In Zoltán Ésik and Zoltán Fülöp, editors, *DLT 2003*, volume 2710 of *Lecture Notes in Computer Science*, pages 279–291. Springer, 2003.

22 Kévin Dubrulle, Véronique Bruyère, Guillermo A. Pérez, and Gaëtan Staquet. Visibly recursive automata. *CoRR*, abs/2603.11648, 2026.

23 Karim Elmaaroufi, Marcell Vazquez-Chanlatte, Stefan Witwicki, Matei Zaharia, and Sanjit A. Seshia. Learning automata from demonstrations, examples, and natural language. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025.

24 Michal Fica and Jan Otop. Active automata learning with advice. In Inês Lynce, Nello Murano, Mauro Vallati, Serena Villata, Federico Chesani, Michela Milano, Andrea Omicini, and Mehdi Dastani, editors, *ECAI 2025 - 28th European Conference on Artificial Intelligence, 25-30 October 2025, Bologna, Italy - Including 14th Conference on Prestigious Applications of Intelligent Systems (PAIS 2025)*, Frontiers in Artificial Intelligence and Applications, pages 1655–1662. IOS Press, 2025.

25 Olivier Gauwin, Anca Muscholl, and Michael Raskin. Minimization of visibly pushdown automata is np-complete. *Log. Methods Comput. Sci.*, 16(1), 2020.

26 John Hewitt, Michael Hahn, Surya Ganguli, Percy Liang, and Christopher D. Manning. Rnns can generate bounded hierarchical languages with optimal memory. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 1978–2010. Association for Computational Linguistics, 2020.

27 Malte Isberner. *Foundations of active automata learning: an algorithmic perspective*. PhD thesis, Technical University Dortmund, Germany, 2015.

28 Malte Isberner, Falk Howar, and Bernhard Steffen. The TTT algorithm: A redundancy-free approach to active automata learning. In *RV 2014*, pages 307–322, 2014.

29 Petr Jancar. Equivalence of pushdown automata via first-order grammars. *J. Comput. Syst. Sci.*, 115:86–112, 2021.

30 Mark Jerrum. The complexity of finding minimum-length generator sequences. *Theor. Comput. Sci.*, 36:265–289, 1985.

31 Xiaodong Jia and Gang Tan. V-star: Learning visibly pushdown grammars from program inputs. *Proc. ACM Program. Lang.*, 8(PLDI):2003–2026, 2024.

32 Bruce Knobe and Kathleen Knobe. A method for inferring context-free grammars. *Information and Control*, 31(2):129–146, 1976.

33 Rick Koenders and Joshua Moerman. Output-decomposed learning of mealy machines. *CoRR*, abs/2405.08647, 2024.

34 Dexter Kozen. Lower bounds for natural proof systems. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 254–266. IEEE Computer Society, 1977.

35 Neil Kulkarni, Caroline Lemieux, and Koushik Sen. Learning highly recursive input grammars. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*, pages 456–467. IEEE, 2021.

36 Viraj Kumar, P. Madhusudan, and Mahesh Viswanathan. Minimization, learning, and conformance testing of boolean programs. In Christel Baier and Holger Hermanns, editors, *CONCUR 2006 - Concurrency Theory, 17th International Conference, CONCUR 2006, Bonn, Germany, August 27-30, 2006, Proceedings*, Lecture Notes in Computer Science, pages 203–217. Springer, 2006.

37 Faezeh Labbaf, Jan Friso Groote, Hossein Hojjat, and Mohammad Reza Mousavi. Compositional learning for interleaving parallel automata. In Orna Kupferman and Pawel Sobocinski, editors, *FoSSaCS 2023*, volume 13992 of *Lecture Notes in Computer Science*, pages 413–435. Springer, 2023.

38 Matteo Magnini, Riccardo Squarcialupi, Martin Tunge Sterri, Ana Ozaki, and Rivera Castillo. Actively learning el terminologies from large language models. In *ECAI 2025*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 1792–1799. IOS Press, 2025.

39 Kurt Mehlhorn. Pebbling moutain ranges and its application of dcfl-recognition. In J. W. de Bakker and Jan van Leeuwen, editors, *Automata, Languages and Programming, 7th Colloquium, Noordweijkerhout, The Netherlands, July 14-18, 1980, Proceedings*, Lecture Notes in Computer Science, pages 422–435. Springer, 1980.

40 Jakub Michaliszyn and Jan Otop. Learning deterministic visibly pushdown automata under accessible stack. In Stefan Szeider, Robert Ganian, and Alexandra Silva, editors, *47th International Symposium on Mathematical Foundations of Computer Science, MFCS 2022, Vienna, Austria, August 22-26, 2022*, LIPIcs, pages 74:1–74:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

41 Joshua Moerman. Learning product automata. In Olgierd Unold, Witold Dyrka, and Wojciech Wieczorek, editors, *ICGI 2018*, volume 93 of *Proceedings of Machine Learning Research*, pages 54–66. PMLR, 2018.

42 Joshua Moerman, Matteo Sammartino, Alexandra Silva, Bartek Klin, and Michal Szynwelski. Learning nominal automata. In *POPL 2017*, pages 613–625, 2017.

43 Yasubumi Sakakibara. Efficient learning of context-free grammars from positive structural examples. *Information and Computation*, 97(1):23–60, 1992.

44 Géraud Sénizergues. The equivalence problem for deterministic pushdown automata is decidable. In *ICALP 1997*, pages 671–681, 1997.

45 Luzi Sennhauser and Robert C. Berwick. Evaluating the ability of LSTMs to learn context-free grammars. In *Proceedings of the Workshop: Analyzing and Interpreting Neural Networks for NLP, BlackboxNLP@EMNLP 2018*, pages 115–124, 2018.

46 Ehud Y Shapiro. Algorithmic program diagnosis. In *Proceedings of the 9th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 299–308, 1982.
47 Frits W. Vaandrager, Bharat Garhewal, Jurriaan Rot, and Thorsten Wißmann. A new approach for active automata learning based on apartness. In *TACAS 2022*, pages 223–243, 2022.
48 Gerco van Heerdt, Tobias Kappé, Jurriaan Rot, and Alexandra Silva. Learning pomset automata. In Stefan Kiefer and Christine Tasson, editors, *ETAPS 2021*, volume 12650, pages 510–530. Springer, 2021.
49 Todd Wareham. The parameterized complexity of intersection and composition operations on sets of finite-state automata. In Sheng Yu and Andrei Paun, editors, *Implementation and Application of Automata, 5th International Conference, CIAA 2000, London, Ontario, Canada, July 24-25, 2000, Revised Papers*, Lecture Notes in Computer Science, pages 302–310. Springer, 2000.
50 Zhengkai Wu, Evan Johnson, Wei Yang, Osbert Bastani, Dawn Song, Jian Peng, and Tao Xie. REINAM: reinforcement learning for input-grammar inference. In Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo, editors, *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, pages 488–498. ACM, 2019.

▶ **Lemma 17.** *Given a **DVPA** $\mathcal{A}$, the relation $\mathcal{B}_{(4)}$ can be computed in polynomial time in the size of $\mathcal{A}$.*

Proof. ◀

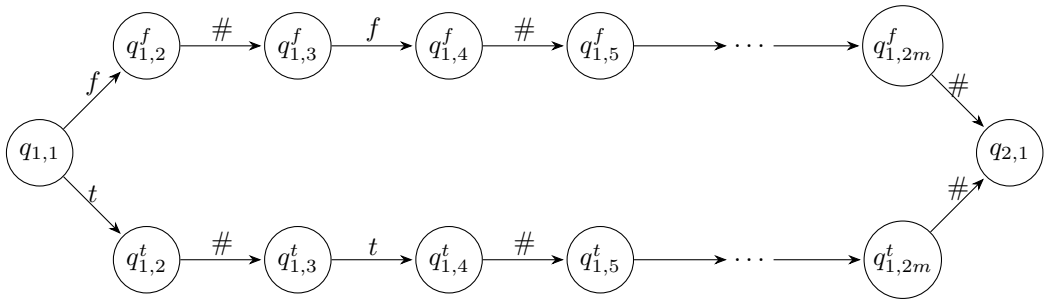
▶ **Theorem 11.** *(1) The problem of deciding, given a rewrite rule $\boldsymbol{\alpha} \to \boldsymbol{\beta}$, a DFA $\mathcal{A}$ and its states q, s_1, s_2, whether (q, s_1, s_2) belongs to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$, is NP-complete. (2) The positive (resp. negative) consistency problem over well-matched rules against a regular language given by a **DFA** is coNP-complete.*

Proof. First, we show NP-hardness of membership to $\mathcal{B}_{(4)}$ via reduction from the 3-SAT problem.

Consider a 3-CNF formula $\varphi = \bigwedge_{1 \leq i \leq k} C_i$. We split every variable x_j in φ into $x_j^1, \ldots, x_j^m$, where m is the number of occurrences of x_j in φ. Let φ^* be obtained from φ by substituting every (propositional) variable x_j with $x_j^1, \ldots, x_j^m$ such that each (propositional) variable in φ^* occurs exactly once. Observe that φ is satisfiable if and only if φ^* has a satisfying assignment, which is consistent among copies of each variable, i.e., $x_j^1, \ldots, x_j^m$ have the same value assigned.

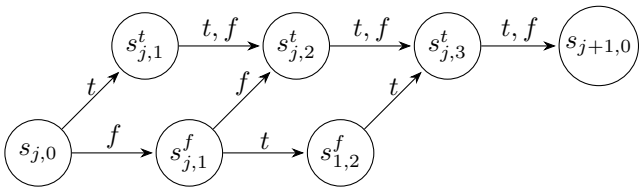
We construct a rule with variables $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ and two **DFA** $\mathcal{A}_1, \mathcal{A}_2$ over $\{t, f, \#\}$ such that variables in $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ correspond to variables in φ^* and for an instantiation of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ $\mathcal{A}_1$ checks that it corresponds to a truth assignment, which is consistent among copies of the variable, an $\mathcal{A}_2$ checks that the truth assignment satisfies φ^*.

Let $\boldsymbol{\alpha}$ be the rule of the from $X_1^1 \# X_1^2 \ldots \# X_1^m \# X_2^1 \# \ldots \# X_n^{m'}$, which consists of all word variables corresponding to propositional variables from φ^* which are grouped so that first are all (variables corresponding to) copies of x_1, next copies of x_2 and so on. There is letter # between any two variables. Then, we define $\mathcal{A}_1$ as in the following diagram, where all transitions that are not depicted here lead to the sink state. Observe that if each variable in $\boldsymbol{\alpha}$ is instantiated to t or f, then $\mathcal{A}_1$ moves from $q_{1,1}$ to $q_{1,n+1}$ if and only if the corresponding truth assignment is consistent among all copies of the variable from φ. If there is an instantiation of $\boldsymbol{\alpha}$ such that $\mathcal{A}_1$ moves from $q_{1,1}$ to $q_{1,n+1}$ over the resulting word, but some of the variables is not instantiated to t or f, then there is (possibly the same) variable



instantiated to a word containing #. Next, we define $\boldsymbol{\beta}$ and $\mathcal{A}_2$ so that in the instantiation no variable contains # and if the corresponding instantiation defines a truth assignment, then it satisfies φ.

Let $\boldsymbol{\beta}$ be the list of variables of all clauses $c_1, \ldots, c_k$ of φ^*, i.e., first are 3 variables from c_1, next 3 variables from c_2 and so on. Now, we define $\mathcal{A}_2$, which is a sequential composition of subautomata $\mathcal{B}_j$ corresponding to clauses $c_1, \ldots, c_k$. Assume that $c_j = y_1 \wedge \neq y_2 \wedge y_3$ and $Y_1 Y_2 Y_3$ is a subword in $\boldsymbol{\beta}$ (the order in $\boldsymbol{\beta}$ is the same). Then, the **DFA** $\mathcal{B}_j$ is illustrated below: All transitions not depicted here lead to the sink state. Consider $\mathcal{A}_2$ to be the **DFA**



obtained from $\mathcal{B}_1, \ldots, \mathcal{B}_k$, where the states $s_{j+1,0}$ is $\mathcal{A}_j$ and $\mathcal{A}_{j+1}$ are merged into a single state, Observe that if there is an instantiation of $\boldsymbol{\beta}$ such that $\mathcal{A}_2$ moves from $s_{1,0}$ to $s_{k+1,0}$, then this instantiation does not contain letter # as it leads to the sink state. Furthermore, if every variable is instantiated to f or t and $\mathcal{A}_2$ moves from $s_{1,0}$ to $s_{k+1,0}$, then every clause of φ^*, and hence it has to satisfy φ^*.

Now, consider an instantiation of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ such that over the instance of $\boldsymbol{\alpha}$ the **DFA** $\mathcal{A}_1$ moves from $q_{1,1}$ to $q_{1,n+1}$ and over the instance of $\boldsymbol{\beta}$ the **DFA** $\mathcal{A}_2$ moves from $s_{1,0}$ to $s_{k+1,0}$, then the instantiation encodes a truth assignment that is consistent among all copies of the same variable from φ and satisfies φ^*. Thus, φ is satisfiable. Conversely, if φ is satisfiable, then the satisfying assignment induces an instantiation of $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ such that $\mathcal{A}_1$ moves from $q_{1,1}$ to $q_{1,n+1}$ and $\mathcal{A}_2$ moves from $s_{1,0}$ to $s_{k+1,0}$. Therefore, $(q_{1,1}, s_{1,0}, q_{1,n+1}, s_{k+1,0})$ belongs to $\mathcal{B}_{(4)}$ of the disjoint union of $\mathcal{A}_1$ and $\mathcal{A}_2$ if and only if φ is satisfiable.

Now, consider $\mathcal{A}$ over $\{t, f, \#, a_1, a_2\}$, which consists of the start state q_0 and disjoint union of $\mathcal{A}_1$ and $\mathcal{A}_2$. In the state q_0 the automaton has self loops over $t, f, \#$ and it moves over a_1 to $q_{1,1}$ and over a_2 to $s_{1,0}$. Finally, in all states of $\mathcal{A}_1$ and $\mathcal{A}_2$ letters a_1, a_2 lead to the sink state. Consider a rule $a_1\boldsymbol{\alpha} \to a_2\boldsymbol{\beta}$, where $\boldsymbol{\alpha} \to \boldsymbol{\beta}$ is defined as above. Then, $(q_0, q_{1,n+1}, s_{k+1,0})$ belong to $\Delta[\boldsymbol{\alpha} \to \boldsymbol{\beta}]$ if and only if φ^* is reachable.

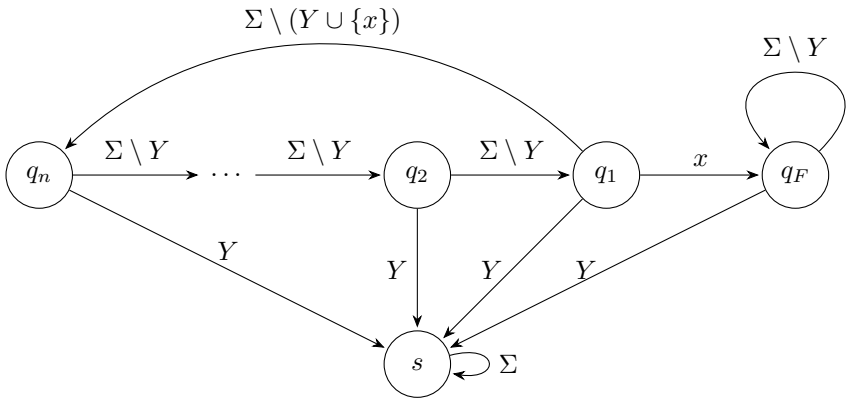
Finally, consider $\mathcal{A}$ defined as above, but we mark $q_{1,n+1}$ and the sink state of $\mathcal{A}_2$ as accepting. If there an instantiation of $a_1\boldsymbol{\alpha} \to a_2\boldsymbol{\beta}$ such that the instance of $a_1\boldsymbol{\alpha}$ is satisfiable and $a_2\boldsymbol{\beta}$ is not, then in $\mathcal{A}_1$ the instance of $\boldsymbol{\alpha}$ leads from $q_{1,1}$ to $q_{1,n+1}$, and in $\mathcal{A}_2$ the instance of $\boldsymbol{\beta}$ leads from $s_{1,0}$ to any other state than the sink, and hence no variable instance consists #. It follows that each variable is instantiated to t or f, and hence over the instance of $\boldsymbol{\beta}$, the automaton $\mathcal{A}_2$ moves from $s_{1,0}$ to either the sink or $s_{k+1,0}$. Therefore, it has to be $s_{k+1,0}$. In summary, if $a_1\boldsymbol{\alpha} \to a_2\boldsymbol{\beta}$ is positively consistent with $\mathcal{L}(\mathcal{A})$, then φ is unsatisfiable.

Conversely, one can see that any satisfying assignment for φ can be transformed to the counterexample to positive consistency. Thus, positive consistency is coNP-hard.

Finally, positive and negative consistency are dual and hence negative consistency is coNP-hard as well. ◀

▶ **Lemma 18.** *The minimal collision problem is NP-complete.*

Proof. Let us fix a parameter n. Before the reduction, we construct a gadget that distinguishes all words of length at most n unless they consist one of the designated letters. Let $Y \subseteq \Sigma$ and $x \in \Sigma \setminus Y$. Consider the automaton $\mathcal{B}_{Y,x}$ depicted below:



Observe that for all words w_1, w_2 of length bounded by n, if $f_{w_1}^{\mathcal{B}_{Y,x}} = f_{w_2}^{\mathcal{B}_{Y,x}}$, then one of the following holds:

- each of the words w_1, w_2 contains at least one letter from Y; in that case f_{w_1}, f_{w_2} are constant functions equal s for every argument,
- the sets positions at which x occurs in w_1 and resp. w_2 are equal; indeed, for any $w \in (\Sigma \setminus Y)^{\leq n}$ we have $f_w(q_i) = q_F$ if and only if $w[i] = x$.

Now, consider $\mathcal{B}_Y$ being the disjoint union of automata $\mathcal{B}_{Y,x}$ over all $x \in \Sigma \setminus Y$.

Observe that for all words w_1, w_2 of length bounded by n, if $f_{w_1}^{\mathcal{B}_Y} = f_{w_2}^{\mathcal{B}_Y}$, then one of the following holds:

- each of the words w_1, w_2 contains at least one letter from Y; in that case f_{w_1}, f_{w_2} are constant functions equal s for every argument,
- words w_1 and w_2 are equal.

Having the automaton $\mathcal{B}_Y$ we reduce the 3-SAT problem to the minimal collision problem. Let φ be a 3-CNF formula over variables $x_1, \ldots, x_n$. Consider $\Sigma = \{x_1, \bar{x_1}, \ldots, x_n, \bar{x_n}, a, b\}$. With each clause c_j we associate the subset C_j of Σ consisting of the literals from c_j. Furthermore, for every variable x_i we define the set $X_i = \{x_i, \bar{x}_i\}$. Now, consider a **DFA** $\mathcal{A}_0$ being the disjoint union of automata $\mathcal{B}_{C_1}, \ldots, \mathcal{B}_{C_m}$ over all clauses of φ and automata $\mathcal{B}_{X_1}, \ldots, \mathcal{B}_{X_n}$ over all variables of φ. We extend $\mathcal{A}_0$ to $\mathcal{A}$ with additional states such that over these states letters from $\Sigma \setminus \{a, b\}$ are self loops, and letters a, b ensure that every component of the disjoint union is reachable. Let k in the instance of the minimal collision problem be equal n, the number of variables.

Now, consider a pair of different words w_1, w_2 such that $f_{w_1}^{\mathcal{A}} = f_{w_2}^{\mathcal{A}}$ and $|w_1|, |w_2| \leq n$. Then, for every automaton $\mathcal{B}_{X_1}, \ldots, \mathcal{B}_{X_n}$, since $w_1 \neq w_2$, each word contains at least one letter from each set X_i. There are n such sets, which are disjoint, and hence w_1 and w_2 contain exactly one letter from each of the sets X_i, i.e., w_1 and w_2 represent consistent

variable assignments (they either contain x_i or $\bar{x}_i$). Furthermore, for every clause c_j each word w_1, w_2 contains at least one letter corresponding to some literal from c_j. Therefore, w_1 and w_2 represent (possibly different) satisfying assignments for φ.

Conversely, assume that φ has a satisfying assignment. Consider two words w_1, w_2 representing this variable assignment with a different order or literals. Then, for every component $\mathcal{B}_{Y,x}$ of $\mathcal{A}_0$, the functions $f_{w_1}^{\mathcal{B}_{Y,x}}, f_{w_2}^{\mathcal{B}_{Y,x}}$ are constant functions equal s. Furthermore, on states in $\mathcal{A}$, which are not in $\mathcal{A}_0$, both functions $f_{w_1}^{\mathcal{A}}, f_{w_2}^{\mathcal{A}}$ coincide with the identity. Consequently, we have (a) $w_1 \neq w_2$, (b) $|w_1| = |w_2| = n$, and (c) $f_{w_1}^{\mathcal{A}_0}, f_{w_2}^{\mathcal{A}_0}$, i.e., the constructed instance of the minimal collision problem is positive. ◀

▶ **Lemma 19.** *The bounded rewriting to a single letter with the parameter k is (a) PSPACE-complete, if k is given in binary, and (b) NP-complete, if k is given in unary.*

Proof. ◀